Lab Report No 3
Implementation of Connected Component Analysis

Digital Image Processing

Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

10th Oct,2024

Department of Computer Engineering,

HITEC University, Taxila

Lab Example No 1:

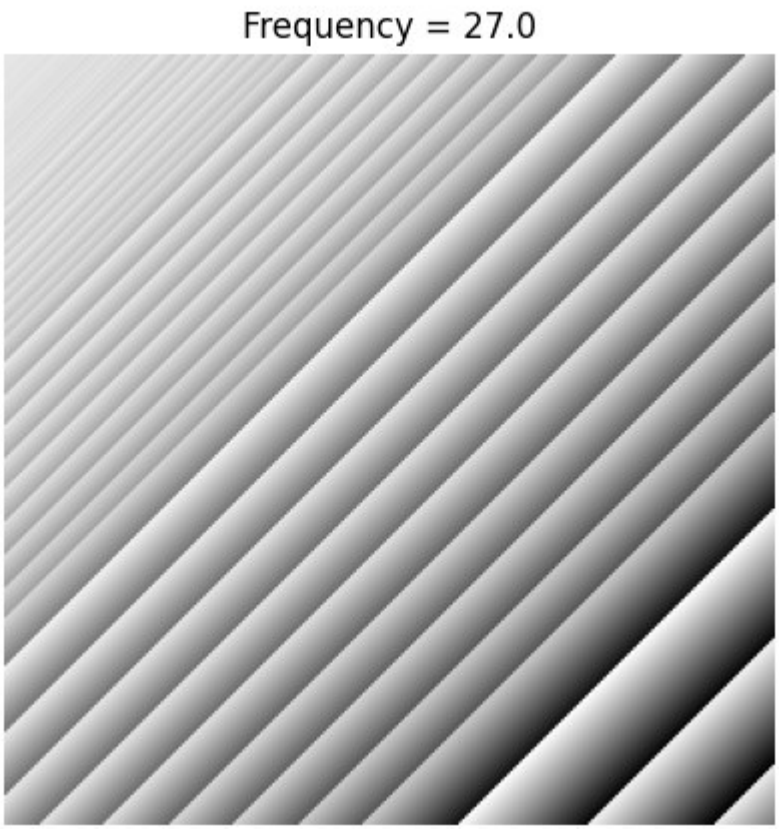
Solution:

Brief description (3-5 lines)
Connected Component Analysis or Labelling enables us to detect different objects from a binary image. Once different objects have been detected, we can perform a number of operations on them: from counting the number of total objects to counting the number of objects that are similar, from finding out the biggest object of the bunch to finding out the smallest and from finding out the closest pair of objects to finding out the farthest etc.
The code

```
import cv2
import numpy as np
# Load the uploaded image
image_path = '/mnt/data/Screenshot 2024-10-10 211014.jpg'
image = cv2.imread(image_path)
# Convert the image to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Apply a binary threshold to the image (convert to black & white)
ret, binary_image = cv2.threshold(gray_image, 127, 255,
cv2.THRESH_BINARY)
# Find contours (connected components)
contours, hierarchy = cv2.findContours(binary_image,
cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
# Create a blank image with the same shape as the original to draw
contours with random colors
image_with_colored_contours = np.zeros_like(image)
# Loop through each contour and draw it with a random color
for contour in contours:
    color = np.random.randint(0, 255, size=3).tolist()  # Generate a
random color
    cv2.drawContours(image_with_colored_contours, [contour], -1,
color, 3)  # Draw contour with random color
# Total number of detected objects (contours)
total_objects = len(contours)
```

```
# Display the results
cv2.imshow('Image with Colored Contours',
image_with_colored_contours)
cv2.waitKey(0)
cv2.destroyAllWindows()
# Print the total number of objects detected
print(f"Total number of objects detected: {total_objects}")
```

The results (Screenshot)



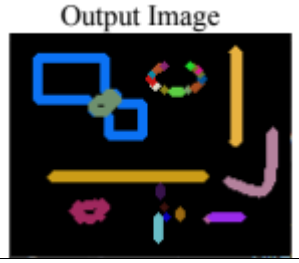
Lab Task No 1:

Solution:

Brief description (3-5 lines)
For the image given below (provided with the lab handout), apply the connectected component labelling and count the total number of colored objects.
The code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image
image_path = '/mnt/data/Screenshot 2024-10-10 211625.jpg'
image = cv2.imread(image_path)
# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Apply thresholding to create a binary image
_, binary = cv2.threshold(gray, 1, 255, cv2.THRESH_BINARY)
# Apply connected component analysis
num_labels, labels = cv2.connectedComponents(binary)
# Display the result
plt.figure(figsize=(10,10))
plt.imshow(labels, cmap='nipy_spectral')
plt.title(f'Total Colored Objects: {num_labels - 1}')  # Subtract 1 to exclude the background
plt.show()
# Print the total number of objects
print(f'Total number of colored objects: {num_labels - 1}')  # Exclude background
```

The results (Screenshot)



Lab Task No 2:

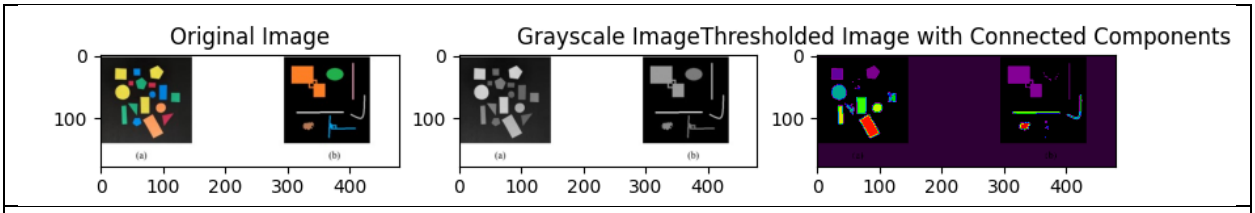
Solution:

Brief description (3-5 lines)
First convert image given below in gray scale and threshold the images and then do connected component analysis. (Do this using build-in commands)

The code

```
import cv2
import matplotlib.pyplot as plt
# Load the image with double backslashes for the Windows path
image = cv2.imread('D:\\BSCOMPUTER ENGINEERING\\4th
YEAR OF COMPUTER ENGINEERING\\7th
SEMESTER\\DIGITAL IMAGE PROCESSING\\PROGRAMS OF
DIGITAL IMAGE PROCESSING\\Screenshot 2024-10-16
192325.jpg')
# Check if the image is loaded properly
if image is None:
    print("Error: Could not load the image. Check the path.")
else:
    # Convert image to grayscale
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # Apply thresholding (binary threshold)
    _, thresholded_image = cv2.threshold(gray_image, 127, 255,
cv2.THRESH_BINARY)
    # Perform connected components analysis
    num_labels, labels_im =
cv2.connectedComponents(thresholded_image)
    # Display the results
    plt.figure(figsize=(10, 5))
    plt.subplot(1, 3, 1)
    plt.title("Original Image")
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    plt.subplot(1, 3, 2)
    plt.title("Grayscale Image")
    plt.imshow(gray_image, cmap='gray')
    plt.subplot(1, 3, 3)
    plt.title("Thresholded Image with Connected Components")
    plt.imshow(labels_im, cmap='nipy_spectral')
    plt.show()
    print(f'Number of connected components: {num_labels}')
```

The results (Screenshot)



Conclusion

In conclusion, Connected Component Analysis (CCA) is a widely used algorithm in image processing tasks like object recognition, shape analysis, and medical imaging. It efficiently segments and labels connected regions within binary images, simplifying further analysis and classification of image structures. CCA's performance can be optimized using two-pass methods or union-find techniques, making it a fundamental tool in digital image processing.